

Unit IV – Prototyping and Testing

Detailed lecture notes — concept visualisation, sketching and mock-ups, low-fidelity and rapid prototyping, testing with users, capturing feedback and the iterative improvement of a design.

Dr. Anantha Raman L., M.E., Ph.D. | Associate Professor, Department of Mechanical Engineering, Rajalakshmi Institute of Technology, Chennai, Tamil Nadu.

SYLLABUS

UNIT IV — PROTOTYPING AND TESTING (6 periods)

Concept visualization. Low-fidelity prototyping. Sketching and mock-ups. Rapid prototyping concepts. Testing methods. User feedback. Iterative design process. Design improvement strategies.

Course Outcome CO4: Develop basic prototypes and test solutions through iterative design methods.

Text: Tim Brown, *Change by Design*, Harper Collins, 2012; Idris Mootee, *Design Thinking for Strategic Innovation*, 1st Edn., John Wiley & Sons, 2013.

References: Nigel Cross, *Engineering Design Methods*, 4th Edn., Wiley, 2008; Jeanne Liedtka and Tim Ogilvie, *Designing for Growth*, Columbia Business School Publishing, 2011 — the chapters on rapid prototyping and learning launch.

CONTENTS

1. Introduction – building in order to think
2. The cost of changing your mind late
3. What a prototype is for
4. What are you prototyping? Role, look and feel, implementation
5. Fidelity – deciding how rough to be
6. Concept visualisation and sketching
7. Storyboards and scenarios
8. Mock-ups and physical models
9. Low-fidelity prototyping in practice
10. Wizard of Oz and service prototypes
11. Rapid prototyping – processes and the digital pipeline
12. Planning and running a user test
13. Capturing and reading feedback
14. The iterative design process
15. Design improvement strategies and knowing when to stop
16. Summary
17. Review questions (two-mark and thirteen-mark)

Learning objectives. After studying this unit you should be able to: (i) explain why designers build

in order to think, and describe how the cost of a change rises with time; (ii) state the question a prototype is meant to answer and classify a prototype by role, look and feel, or implementation; (iii) choose an appropriate fidelity and justify the choice; (iv) visualise a concept through sketching, storyboarding and scenario writing; (v) build a low-fidelity prototype from paper, card and found materials, and stage a Wizard of Oz test; (vi) describe the common rapid-prototyping processes and the CAD-to-part pipeline, with their capabilities and limits; (vii) plan and moderate a user test, capture feedback without contaminating it, and read the results; and (viii) run the build-measure-learn loop and decide when to iterate, when to pivot and when to stop.

1. Introduction – building in order to think

Unit III ended with two or three concepts and a matrix explaining why they were chosen. Nothing in that matrix is evidence. Every cell in it is a judgement made by people sitting in a room, about a solution that does not yet exist, for users who were not present. The purpose of this unit is to replace as many of those judgements as possible with observations, and the instrument for doing so is the prototype.

The habit a first-year student brings from school is to think first and build afterwards: the design is settled in discussion, and making is the last step, an act of execution. Design thinking reverses this. Making is part of the thinking. A sketch forces a decision about proportion that a paragraph allows you to dodge; a cardboard model settles in four minutes an argument about reach that had run for twenty; a paper screen shown to one student reveals that the word everyone on the team understood means nothing to anybody else. The phrase used for this in studios is **build to think**, and it is meant literally.

You do not build a prototype because you have decided. You build one because you have not.

Three consequences follow, and they organise the whole unit. First, if the prototype exists to answer a question, it need only be as good as that question requires — which is usually very rough. Second, a prototype has to meet a user, because the questions worth asking are about somebody else's behaviour and cannot be answered by the team. Third, the result of the test is almost never a verdict on the whole concept; it is a list of corrections, which means the process loops. Roughness, users and looping are the three themes of Unit IV.

2. The cost of changing your mind late

The economic argument for prototyping early is the strongest one available, and it is worth giving to engineering students in its own terms. The cost of changing a design is not constant over the life of a project. It rises steeply, and it rises for reasons that have nothing to do with anybody's competence.

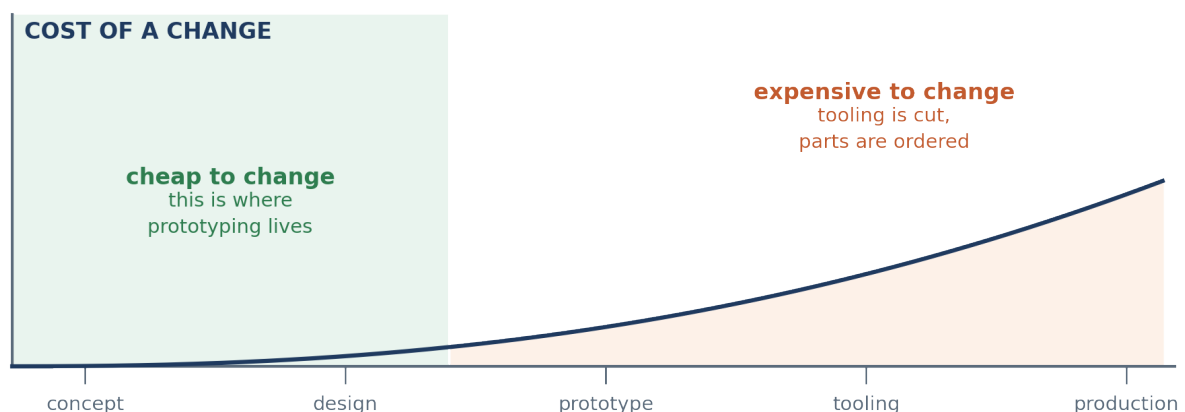


Figure 4.1 The cost of a change against the stage at which it is made. The curve is why design thinking front-loads the learning.

In the sketch phase a change costs a fresh sheet of paper. After the concept is frozen it costs a re-analysis. After tooling it costs a new mould. After launch it costs a recall, and after that it costs a reputation. The usual figures quoted in industry put a change during detailed design at ten times the cost of the same change during concept design, and a change after production at a hundred to a thousand times. The precise multiplier varies by industry; the shape of the curve does not.

There is a second curve running the other way, and the pair together make the case. Early in a project you have the greatest freedom to change the design and the least knowledge about whether you should. Late in a project you have the knowledge and no freedom left to use it. Prototyping is the deliberate purchase of knowledge at the point in the project where it is still cheap to act on, and the price paid is a few hours of making something that will be thrown away.

KEY POINT A prototype is thrown away by design. A team that will not discard a prototype has stopped using it as an instrument and started defending it as a product — which is the exact failure the technique exists to prevent.

Stage	What a change costs	What you still know nothing about
Sketch	A sheet of paper and ten minutes	Almost everything — which is why sketching is cheap and fast
Low-fidelity prototype	An hour and some card	Materials, cost, durability — but you now know whether the idea makes sense
Working prototype	A few days and some material cost	Manufacture at volume, long-term wear
Tooling and pilot production	Lakhs of rupees and weeks of lead time	Field behaviour over years
After launch	Recall, redesign and the customer's trust	Very little — and it is now extremely expensive to act on what you have learnt

3. What a prototype is for

A prototype is any representation of an idea, made before the idea is finished, that lets somebody experience or examine part of it. That definition deliberately includes a sketch, a role play, a spreadsheet, a cardboard box with a screen drawn on it and a fully working rig. What makes any of them a prototype rather than a craft object is that each is built to answer a question.

- **To answer a question that argument cannot settle.** Two members disagree about whether the handle can be reached with one hand. Building the handle in cardboard ends the disagreement in four minutes, and the answer is not a compromise between the two opinions.
- **To find the flaw while it is still free to fix.** Faults are found by use, not by inspection. The earlier a prototype is used, the cheaper the fault.
- **To communicate to people who do not read drawings.** A user, a funder or a village committee cannot react to an assembly drawing. They can react immediately to something they can hold or watch.
- **To obtain permission and resources.** A working rough model is far more persuasive than a slide deck, because it shifts the question from whether the idea could work to what should be improved.

- **To learn what the team itself thinks.** A surprisingly common outcome is that the team looks at its own prototype and discovers it does not want the thing it has been arguing for.

A prototype is an experiment with a question attached. No question, no prototype.

The rule in that sentence has a practical form that should be enforced in the laboratory: before any material is cut, the team writes on the top of the sheet the question this prototype will answer, and the observation that would count as an answer either way. “Can a user carry this up two flights of stairs without setting it down?” is a question. “Is this a good design?” is not — nothing that happens in the test could answer it.

4. What are you prototyping? Role, look and feel, implementation

The most useful single framework in this unit comes from a 1997 paper by **Stephanie Houde and Charles Hill** at Apple, titled *What do Prototypes Prototype?* Their observation was that arguments about whether a prototype is any good are usually arguments between people judging it on different dimensions. They proposed three.

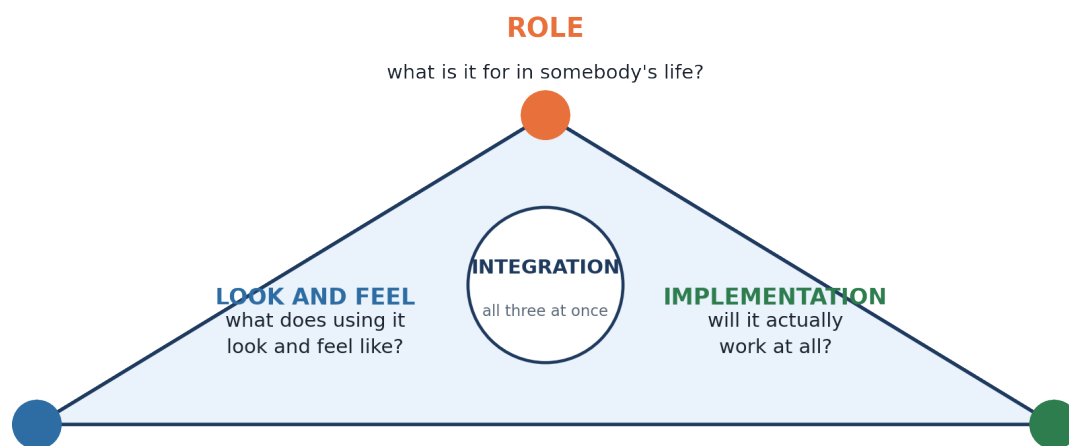


Figure 4.2 Houde and Hill's model. Name the dimension you are prototyping before you choose materials; an integration prototype attempts all three and is expensive.

Dimension	The question it answers	Typical prototype
Role	Does this belong in the user's life at all? What is it for?	Storyboard, scenario, role play, a fake service run by hand
Look and feel	What is it like to see, hold and use?	Sketch, foam or card model, click-through of paper screens
Implementation	Can it be made to work at all?	Breadboard circuit, mechanism rig, throwaway code — appearance irrelevant
Integration	Does the whole thing hold together in real use?	A prototype attempting all three; expensive, so built late and rarely

The discipline the model imposes is to **name the dimension before building**. A beautifully finished model that cannot answer whether anybody wants the product has prototyped look and feel when the open question was role. A working mechanism presented to a user who then complains that it is ugly has been judged on the wrong axis — the correct response is that its appearance is not what is being tested, and the team should have said so before handing it over.

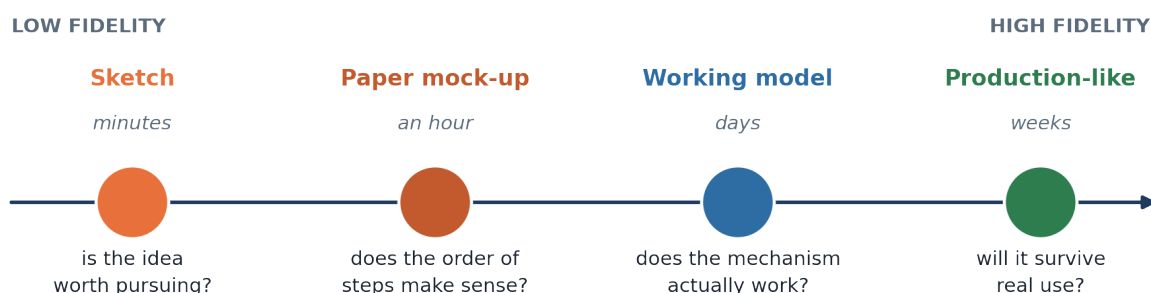
KEY POINT Most failed prototypes tried to answer all three questions at once and answered none of

them well. One prototype, one dimension, one question.

IN CLASS Fifteen minutes. Take the concept your team selected at the end of Unit III. Write down the three most uncertain things about it. Classify each as role, look and feel or implementation, and decide which single one you will prototype in the next session. Write the question and the observation that would answer it at the top of your sheet.

5. Fidelity – deciding how rough to be

Fidelity is how closely a prototype resembles the finished product. It is not a measure of quality and a high-fidelity prototype is not a better prototype; it is a more expensive one, and it buys a different kind of answer. Fidelity is chosen, and the choice should be defended.



go only as high as the question needs — fidelity you did not need is time you did not have

Figure 4.3 The fidelity spectrum. Cost, time and the confidence of an answer all rise together; so does the reluctance to throw the prototype away.

	Low fidelity	High fidelity
Made from	Paper, card, tape, found objects, role play	Machined or printed parts, working code, real materials
Time	Minutes to a few hours	Days to weeks
Answers	Role and broad look and feel	Implementation, detail, ergonomics
Feedback it draws	Bold, structural: "why would I use this at all?"	Narrow, cosmetic: "the corner is too sharp"
Risk	Users cannot imagine the finished thing	Team becomes attached; users hesitate to criticise something polished
Use it when	The concept itself is uncertain	The concept is settled and the details are not

The middle two rows contain the point that students find counter-intuitive. A rough prototype invites rough criticism. A user handed a torn-paper sketch will say that the whole idea is wrong, because it plainly cost nobody very much; handed a polished 3D print of the same idea, the same user will politely suggest rounding a corner. The technical name for the second case is the **politeness effect**, and it is why a team that polishes too early stops learning anything structural.

Roughness is not a compromise forced on you by the timetable. It is what gives you

permission to be told that the idea is wrong.

- **Match fidelity to the question.** Role questions are answered at the lowest fidelity there is — a story told aloud. Implementation questions need something that actually runs, though it may be hideous.
- **Raise fidelity on one axis only.** A prototype can have a realistic mechanism and a cardboard body, or a finished shell and no working parts. Raising everything together is how weeks disappear.
- **Keep it cheap enough to destroy.** If you would hesitate to cut it in half in front of the class, it has become too precious to be useful.
- **Say what it is before you hand it over.** “The shape is not being tested today; tell me about the sequence.” A single sentence redirects the feedback you receive.

6. Concept visualisation and sketching

Concept visualisation is the act of getting an idea out of one person's head and onto a surface where the team can argue about the same thing. Until that happens, each member is defending a private mental picture, and the discussion is longer and less productive than anybody realises. The cheapest and fastest instrument for it is a pen.

6.1 Sketching is not drawing

The most common objection in a first-year class is that the student cannot draw. This confuses two different activities. **Drawing** is a representational skill aimed at a finished image and judged on how it looks. **Sketching** is a thinking tool aimed at a decision and judged on whether it settles one. A design sketch needs to be fast, disposable and clear enough to point at. Everything in a useful design sketch can be made from six primitives and a little practice.

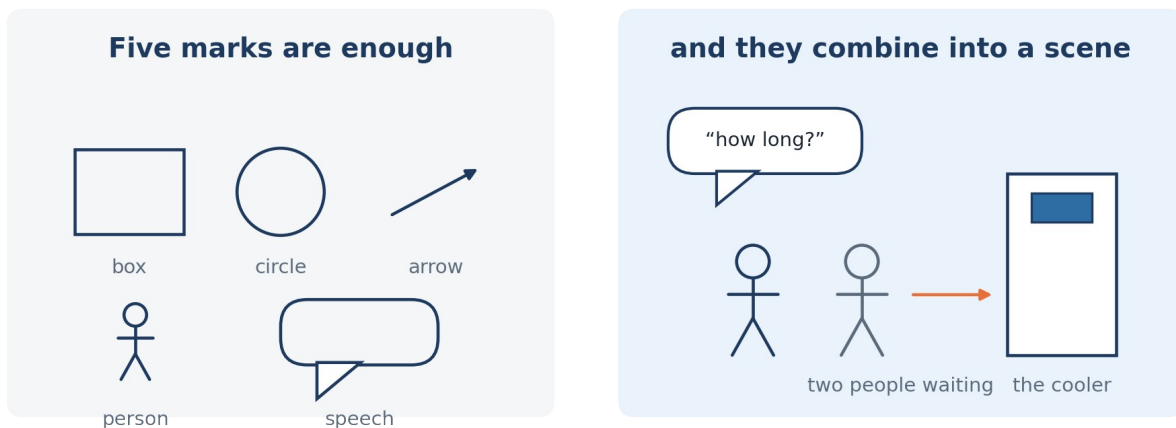


Figure 4.4 The vocabulary of a design sketch. Squares in perspective, cylinders, arrows for motion, a human figure for scale, callouts and a crop box are enough for almost any concept.

- **Draw big and draw fast.** Small, careful sketches invite small, careful thoughts. Use a marker rather than a pencil so that you cannot erase.
- **Always put a human or a hand in it.** A concept drawn without a person has no scale, and scale is where most concepts fail.
- **Label it.** Two words beside a line are worth more than another ten minutes of shading. A sketch nobody can read is a doodle.
- **Draw the same idea several times.** Six quick variants beat one careful drawing, because the variants are what lets you choose.
- **Date and keep every sheet.** The rejected sketches are the record of why the design is as it is, and are worth marks in a design review.

6.2 Other ways to visualise a concept

Instrument	What it shows	When to use it
Concept sketch	Form, arrangement, scale	At the very start, in quantity
Exploded view	How parts relate and assemble	When the argument is about assembly or maintenance
Flow or sequence diagram	The order of steps and the decision points	For services and software, where there is no object to draw
Journey map	The user's experience over time, with highs and lows	When the concept changes an experience rather than an object
Wireframe	Layout and hierarchy of a screen, without styling	For any digital interface, before any colour is chosen
Storyboard	The situation before, during and after use	When the open question is role

IN CLASS Twenty minutes. Individually and in silence, sketch six variants of your team's concept, one minute each, each with a human figure for scale and at least two labels. Pin all sheets up together. The team then marks the three features worth carrying forward, taken from any of the sheets.

7. Storyboards and scenarios

A storyboard is a short strip of frames showing a user's situation before, during and after the use of the concept. It is borrowed from film, and it is the standard prototype for the **role** dimension: it tests whether the thing belongs in the user's life at all, which no object can answer by itself. It is also the cheapest prototype in existence — six boxes on one sheet of paper.

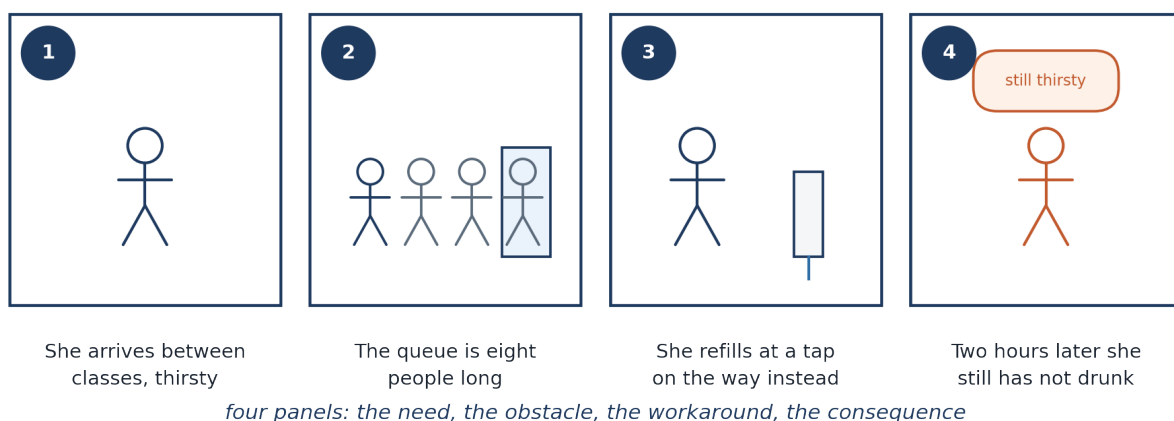


Figure 4.5 A six-frame storyboard. The concept appears only in the middle frames; the first and last frames are what make the strip an argument.

1. Frame 1 — the user and the setting, with no concept in sight. Who is this, where are they, what are they trying to do?
2. Frame 2 — the problem as it occurs today, shown happening rather than described.
3. Frame 3 — the moment the concept enters. How does the user come across it? A concept nobody would ever encounter has already failed.
4. Frame 4 — the concept in use, drawn from the user's point of view.
5. Frame 5 — the outcome. What is now different for this person?

6. Frame 6 — afterwards. What happens the next day, the next week, or to the person who has to clean or maintain it?

KEY POINT The first and the last frames do the work. A storyboard that begins with the product and ends with a satisfied smile is an advertisement. A storyboard that begins with a frustrated person and ends with a consequence is a design argument.

A **scenario** is the same thing written rather than drawn: a short narrative, half a page at most, naming a specific person from the team's field work, a specific occasion and a specific outcome. It is used where a storyboard would be slow, and it has one great advantage — it can be read aloud to a user in thirty seconds, and their face while they listen is data. Scenarios are written in the past or present tense about one named person, never in the conditional about users in general; *“Kavitha reaches the stop at 7.40 and cannot tell whether the bus has already gone”* is a scenario, whereas *“users would be able to check bus timings”* is a feature list.

IN CLASS Twenty-five minutes. Draw a six-frame storyboard for your concept using the structure above, and write a half-page scenario naming one real person from your Unit II field work. Read the scenario aloud to a member of another team and record the first question they ask.

8. Mock-ups and physical models

A mock-up is a three-dimensional model built to answer questions that a flat sheet cannot: how big is it, where does the hand go, will it fit through the door, does it tip over. It need not work and it need not be accurate in any dimension that is not under examination. The mechanical engineering student's instinct is to model it in CAD first; the studio instinct is to make it in ten minutes from card and find out that the whole proportion is wrong before any CAD time is spent.

Kind of model	Made from	What it settles
Blocking model	Foam board, stacked card, a cereal box	Overall size, presence in a space, whether it is carryable
Ergonomic mock-up	Card, tape, modelling clay, a broom handle	Reach, grip, height, angle of use, one-handed operation
Appearance model	Sanded foam, painted card, 3D print	Form and finish only — it has no working parts
Mechanism rig	Meccano, scrap, laser-cut acrylic, motors	Whether the movement works; appearance is irrelevant
Full-size stand-in	Chairs, string, chalk on the floor, a doorway taped out	Layout of a space or a queue; costs nothing at all

KEY POINT Full size beats accurate. A full-size model that is wrong in every dimension but one still answers the question about that dimension; a beautiful quarter-scale model answers none of the questions that involve a human body.

9. Low-fidelity prototyping in practice

Low-fidelity prototyping is the core practical skill of this unit and the one examined in the laboratory. The toolkit is deliberately poor: paper, card, tape, string, rubber bands, straws, foil, clay, bottle caps, scissors

and markers. Nothing on that list requires a workshop, an approval or a purchase, which means the barrier between having an idea and testing it is under ten minutes.

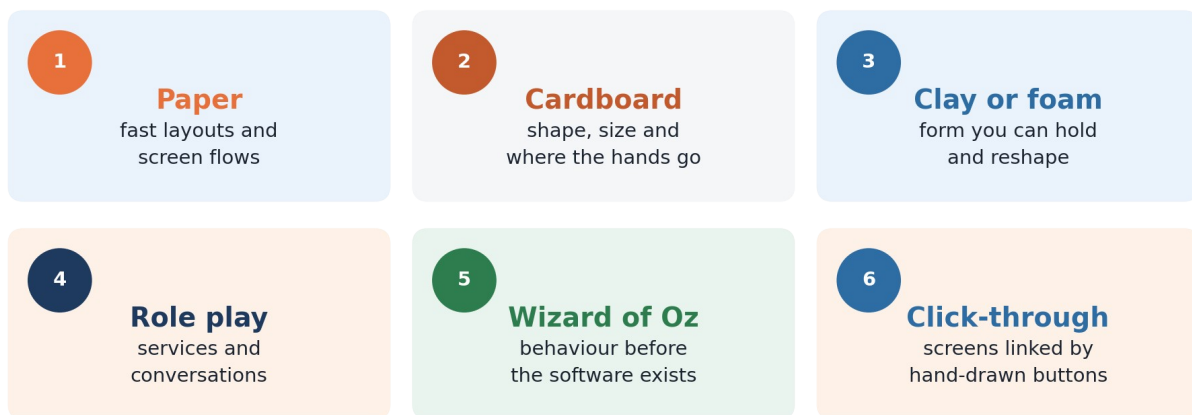


Figure 4.6 The low-fidelity toolkit. Everything here is cheap enough that a failed prototype costs nothing but the time to make the next one.

9.1 The method

1. Write the question on the sheet first, with the observation that would answer it.
2. Set a timer. Thirty minutes is generous for a first prototype and the limit is part of the method, because it forces the team to build only what the question needs.
3. Build the part that the question touches and fake everything else. A screen that never changes can be drawn; a motor that is not being tested can be a hand.
4. Build it full size wherever a human body is involved.
5. Stop when the prototype can be used by somebody who was not in the room. That is the finishing condition — not when it looks finished.
6. Put it in front of a user the same day. A prototype that is not tested on the day it is built usually becomes a decoration.

9.2 Paper prototyping for interfaces

For anything with a screen, the standard low-fidelity method is the paper prototype. Each screen is drawn on a separate card, menus and dropdowns are drawn on small slips that can be laid on top, and a member of the team acts as the computer: when the user touches a drawn button, the human computer swaps the card. The human computer may not speak, explain or help, because a real interface will not, and every moment of hesitation is a finding. Ten screens take twenty minutes to draw and reveal, reliably, that two or three of the labels mean nothing to anyone outside the team.

If the user hesitates, the interface is wrong. The hesitation is the finding, and no explanation from the team can be part of the product.

9.3 Common faults in low-fidelity work

- **Building the whole thing.** The team builds every feature at low fidelity and spends two sessions doing it. Build only what the question touches.
- **Decorating.** Colouring, neat lettering and a printed title page add hours and subtract honest feedback.
- **Refusing to test until it is ready.** It is never going to be ready. Testing it while it is embarrassing is the entire point.

- **Explaining instead of observing.** The moment the team starts explaining the prototype to the user, the test has ended and a sales pitch has begun.
- **Keeping only the successes.** The prototype that failed is the one that taught you something; photograph it before it goes in the bin.

IN CLASS Forty minutes, in the laboratory. Build a full-size low-fidelity prototype of the single dimension you named in Session 1. Write the question on the sheet before you start and stop at the timer. Then swap teams: build nothing further, but let another team use your prototype while you watch in silence.

10. Wizard of Oz and service prototypes

Some concepts cannot be prototyped as objects, because the thing being designed is a service, a system or a piece of intelligence that would take months to build. The technique for these is to fake the machinery and run the experience for real. It is called a **Wizard of Oz** prototype, after the man behind the curtain operating the levers, and it was made a standard design method by teams at IBM and later at IDEO; **Tom Kelley** describes it in *The Art of Innovation* as a favourite device of the studio.

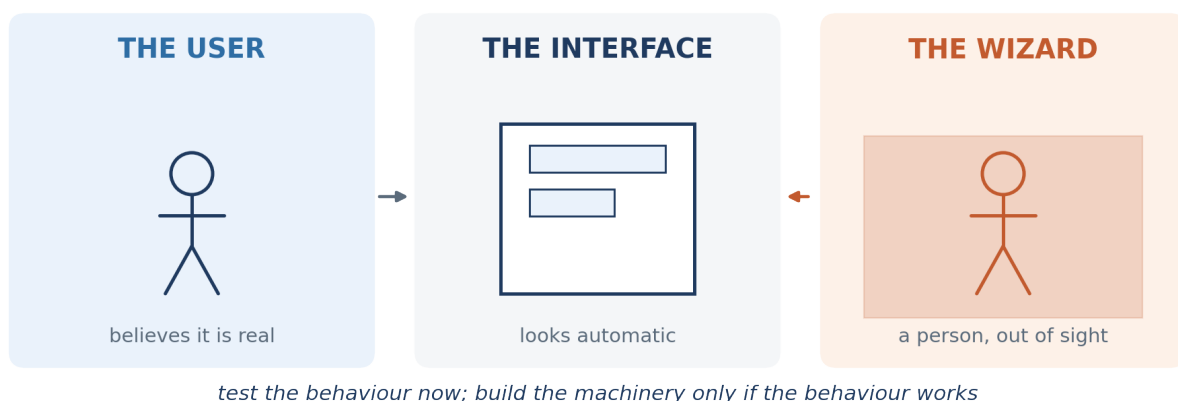


Figure 4.7 A Wizard of Oz test. The user believes they are using a system; a hidden team member produces every response by hand.

The user is told that they are using a working system. They speak to it, type into it or press its buttons. Out of sight, a member of the team reads what the user did and produces the response by hand — typing the reply, sliding the next card into view, moving the object. Because the user believes it is real, their behaviour is real, and that behaviour is what the team needs. A speech assistant, a recommendation system, an automated gate and a help desk can all be tested this way in an afternoon, long before a line of code exists.

Concept to test	What the wizard does	What you learn in one afternoon
Voice assistant for a lab	Listens and types replies that appear on the user's screen	What people actually ask for, and in what words
Automatic attendance at a door	Watches and triggers the green light manually	Whether people queue, crowd or tailgate
Recommendation feature	Chooses the suggestions by hand from a list	Whether the suggestion is noticed at all, and when it annoys
Campus delivery service	Team members carry parcels themselves for two days	Real demand, real timings, real complaints

KEY POINT The point of a Wizard of Oz test is not to deceive the user for its own sake but to obtain honest behaviour. Debrief every participant afterwards, tell them the system was operated by a person, and ask permission to keep their data. That debrief is not optional.

11. Rapid prototyping – processes and the digital pipeline

Rapid prototyping is the name given to a family of processes that turn a digital model into a physical part in hours rather than weeks, without tooling. It arrived in the late 1980s with **Chuck Hull's** stereolithography, patented in 1986, and **Scott Crump's** fused deposition modelling, patented in 1989, and it changed the economics of the middle of the design process: a part that would once have required a pattern maker and a fortnight can now be in a student's hand the next morning.

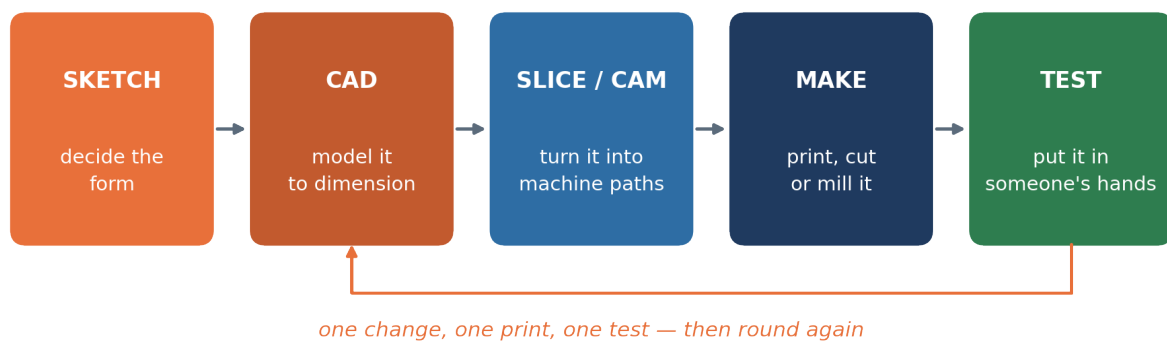


Figure 4.8 The digital pipeline. Each arrow is a place where information can be lost, and the loop back from the printed part to the CAD model is the useful part.

1. Model the part in CAD, or capture an existing object with a 3D scanner.
2. Export a mesh — STL is still the common format, and it approximates every curved surface with flat triangles, so the export resolution matters.
3. Slice the mesh into layers and generate the machine path, choosing layer height, infill, orientation and supports. Most print failures are decided here.
4. Build the part layer by layer on the machine.
5. Post-process: remove supports, cure, sand, sometimes join sub-parts.
6. Test it, mark it up by hand, and return to the CAD model. The loop is the reason the process exists.

Process	How it works	Suited to
FDM / FFF	A thermoplastic filament is extruded layer by layer	Cheap functional parts, jigs, housings; the standard college machine
SLA / DLP	A liquid resin is cured by a laser or projector	Fine detail, smooth surfaces, small parts; brittle and needs curing
SLS	A laser fuses powdered polymer, unsintered powder supporting the part	Strong parts with complex internal geometry and no supports
Metal (DMLS/SLM)	A laser fuses metal powder layer by layer	End-use metal parts; expensive, and rare in an undergraduate laboratory
Laser cutting	A laser cuts flat sheet, parts then slotted together	Very fast planar prototypes — often faster and better than printing

Process	How it works	Suited to
CNC machining	Material is removed from a solid block	Accurate parts in real production materials; subtractive, not additive



Figure 4.9 Three additive processes compared. All three build in layers; they differ in the material state and in how the layer is consolidated.

Two cautions belong beside the enthusiasm. First, **rapid is relative**: a print that takes nine hours is slower than a cardboard model that takes nine minutes, and for role and look-and-feel questions the cardboard usually wins. Printing is for parts whose geometry genuinely has to be right. Second, **a printed part is not the production part**. Its anisotropic layer bonding, its surface finish and its material properties differ from an injection moulding, so it will answer questions about fit and geometry far better than questions about strength or life.

Print the part whose geometry must be right. Everything else is faster in card, and the team learns more from it.

12. Planning and running a user test

A prototype that has not met a user has told the team nothing it did not already believe. Testing is where the unit's argument closes: the question written at the start of the build is now answered by somebody else's behaviour. It is also, reliably, the part a student team does worst, because the natural instinct is to present rather than to observe.

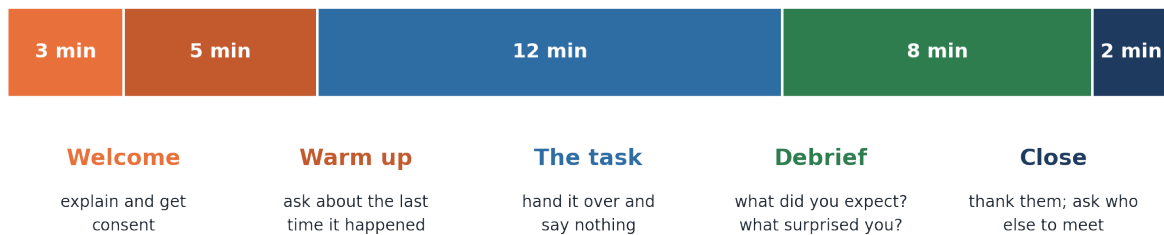
12.1 How many users

The usual objection is that five users cannot be representative. They are not meant to be. **Jakob Nielsen and Thomas Landauer** showed in 1993 that usability problems are found according to a curve of diminishing returns: the first user finds about a third of the problems, five users find roughly four-fifths, and the tenth user finds almost nothing the first five did not. The conclusion is not that five is a magic number but that it is far better to run three tests of five users with a redesign between them than one test of fifteen. Testing is not a survey; you are hunting for faults, not measuring a population.

KEY POINT Five users, then fix, then five more. A larger single test buys you better statistics about a design you are about to change anyway.

12.2 Preparing the session

Thirty minutes with one user



the twelve silent minutes in the middle are the whole point of the session

Figure 4.10 The roles in a test session. Separating the moderator from the note-takers is what makes the record usable afterwards.

1. Write the question again at the top of the script. Everything in the session serves it.
2. Recruit users who match the people in your field work, not your friends. A classmate who has heard the idea discussed for three weeks is useless as a participant.
3. Write two or three realistic tasks rather than a list of features. "You have arrived at the hostel at nine in the evening and need to report a leaking tap" is a task; "try out the reporting screen" is not.
4. Assign roles: one moderator who speaks, one or two note-takers who do not, and a wizard if the prototype needs one.
5. Prepare the opening words, including permission to record, the reminder that the prototype is being tested and not the user, and the request to think aloud.
6. Decide in advance what you will measure: where they hesitate, what they do first, what they say aloud, what they do that you did not anticipate.

12.3 Moderating

Do	Do not
Say: "this is rough, and we want to know what is wrong with it"	Explain how it is supposed to work before they touch it
Hand it over and stay quiet; count to ten before saying anything	Rescue the user the moment they hesitate
Ask "what are you thinking now?" and "what did you expect to happen?"	Ask "do you like it?" or "would you use this?"
Let silence run; it is where the interesting behaviour happens	Fill the silence with a description of the next feature
Note exactly what they did, in their words, with the timing	Note what they meant, or what you think they should have done
Thank them and ask what one thing they would change	Argue with a criticism, or explain why they are mistaken

You are testing the prototype, not the user. Every time you explain something, you have removed a finding and replaced it with a compliment.

13. Capturing and reading feedback

Feedback captured badly is worse than no feedback, because it carries the authority of the field while consisting largely of the team's own opinions. The rule from Unit II carries over unchanged: keep what happened separate from what you think it means. The standard instrument is the **feedback capture grid**, a sheet divided into four quadrants that the whole team fills in during and immediately after each session.

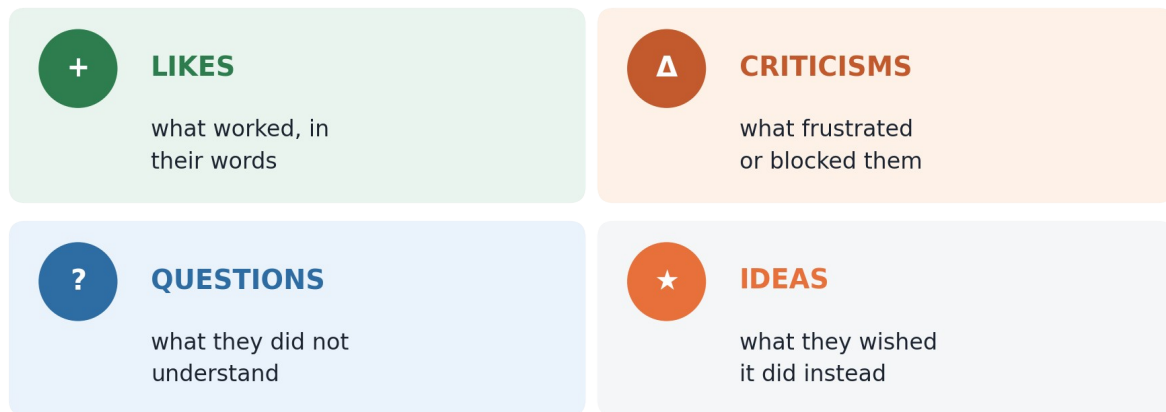


Figure 4.11 The feedback capture grid. Every observation lands in one of the four quadrants, and the two lower quadrants are where the next prototype comes from.

Quadrant	What goes in it	Example
Likes — what worked	Things the user did easily, or praised without prompting	"Found the report button without looking at anything else"
Criticisms — what did not work	Hesitations, errors, complaints, abandoned attempts	"Tried to press the photograph and nothing happened; gave up after twice"
Questions — what it raised	Things the team now does not know and must find out	"Does anyone report a fault at night, or do they wait for morning?"
Ideas — what it suggested	New ideas prompted by the session, including the user's own	"She suggested a sticker on the tap itself with the room number on it"

- **Capture during, not after.** Memory reorganises a session into a story within about twenty minutes, and the story is always more flattering than the session.
- **Record verbatim.** "It is confusing" is not usable. "I thought the green one was the one I already sent" is.
- **Count behaviour, not opinions.** Four of five users hesitated at the same step is a finding. Four of five said it was nice is politeness.
- **Look for the unexpected use.** A user doing something you did not design for is the most valuable event in a test session.
- **Debrief within the hour.** Each team member writes their three biggest findings alone before anybody speaks, so that the loudest member does not fix the team's memory.

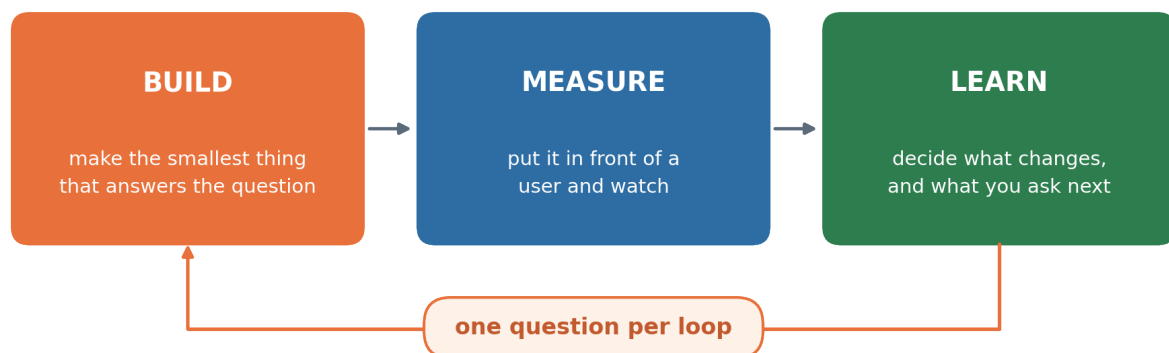
Reading the grid afterwards is a matter of looking for repetition and for surprise. A criticism that appears in one session out of five is an individual; the same criticism in four sessions out of five is a design fault. A single surprise — one user doing something nobody predicted — is worth investigating even if it happened only once, because it usually means the team has misunderstood the situation rather than the interface.

IN CLASS Fifty minutes. Test your prototype with five users from outside the class. One moderator,

two note-takers, three tasks, no explaining. Fill a feedback capture grid during each session. Afterwards, list the findings that appeared in three or more sessions and the single most surprising thing that happened.

14. The iterative design process

Iteration is the feature that distinguishes design thinking from the linear engineering process a student has already met. In a linear process each stage is completed, signed off and left behind, and returning to an earlier stage is a failure to be explained. In an iterative process returning is the method: each loop is cheaper than the mistake it prevents, and a team that never returns has merely postponed its learning until the expensive end of the curve.



the loop is the method — a design that never looped was never tested

Figure 4.12 The build-measure-learn loop. The loop is entered as often as the budget allows, and each turn should be faster than the last.

1. Build the smallest thing that can answer the current question.
2. Measure what users do with it, not what they say about it.
3. Learn: state in one sentence what you now know that you did not know before.
4. Decide: change the prototype, change the concept, or change the problem. Then write the next question and go round again.

The fourth step contains the decision students find hardest. A test result can send a team back to any of the earlier stages, and recognising which is the skill. A confusing label sends you back to the prototype. A concept nobody wants sends you back to ideation. A concept that solves a problem users do not have sends you back to empathy, and that is not a disaster — it is the process working, and working early.

What the test showed	Go back to	Because
Users stumbled on one step	This prototype	The concept is sound; the execution is not
Users liked it but would not use it	Ideation	The concept does not fit the life around it
Users did not have the problem at all	Empathy and definition	The problem was framed from assumption, not observation
Users solved it their own way instead	Empathy and definition	There is an existing workaround you did not find in the field
It worked, and revealed a bigger problem next to it	Definition	The opportunity has moved; reframe and continue

KEY POINT Each iteration should be faster and narrower than the one before. If loop three takes longer than loop one, the team has stopped testing and started building the product.

15. Design improvement strategies and knowing when to stop

A list of findings is not a plan. The last skill of the unit is turning a grid full of observations into an ordered set of changes, making them, and recognising the point at which further iteration stops paying.

15.1 Ordering the changes

Findings are sorted on two axes — how many users it affected and how much it costs to fix — and worked in that order. Fix the frequent and cheap faults first, because they are free improvements; schedule the frequent and expensive ones, because they are the real design work; and be honest that the rare and expensive ones may never be worth fixing at all. A team that begins with the most interesting fault rather than the most common one spends its week well and improves the design very little.

Strategy	What it means	When it applies
Fix	Correct a fault while keeping the concept	Frequent, specific, and the concept survived the test
Simplify	Remove a feature, a step or a control	Almost always; removal improves more designs than addition
Combine	Merge two elements users treated as one	Users repeatedly moved between two parts to do one job
Reframe the interaction	Change how the user is asked to do it, not what	Users understood the goal but not the route to it
Pivot	Keep the insight, change the solution	The need is real and the concept does not serve it
Abandon	Drop the concept and return to the shortlist	Users did not have the problem; nothing downstream will fix that

15.2 Simplify before you add

The commonest response to a criticism is to add something: a label, a button, a warning, an instruction sheet. It is also usually the wrong one. An instruction sheet is an admission that the design failed to explain itself, and each addition makes the next fault harder to see. The professional habit is to ask first what can be removed — and a great deal of what a student team builds into a first prototype turns out to be there because the team found it interesting rather than because any user needed it.

If the prototype needs an instruction sheet, the instruction sheet is the bug report.

15.3 When to stop

Iteration does not converge by itself; it stops when somebody decides. Four honest stopping conditions are worth teaching, and a fifth dishonest one worth naming.

- **The findings have gone quiet.** A round of five users produces nothing that the previous round did not. The design is stable at this fidelity — which may still mean it is time to raise the fidelity and test again.

- **The remaining faults are outside the scope.** What is left needs money, authority or a manufacturing decision that the project does not control. Record them and hand them on.
- **The question is answered.** The prototype existed to settle one thing. It has settled it. Build the next prototype for the next question rather than polishing this one.
- **The budget is spent.** Time and material are finite; a team that acknowledges this and documents what it would test next is behaving professionally, not failing.
- **The dishonest one: the team is tired of being criticised.** This is the usual real reason, and it arrives shortly after the prototype starts looking good. Notice it and name it.

KEY POINT Stopping is a decision recorded with a reason, exactly like every other decision in this course. “We stopped because two consecutive rounds produced no new findings” is a defensible line in a report; “we ran out of time” without a list of what remains is not.

IN CLASS Thirty minutes. Sort your findings by frequency and by cost of fixing. Choose the three changes you will make, state for each which strategy from the table you are applying, and rebuild only those parts. Then write the one question your next prototype would answer if the course allowed a fourth loop.

16. Summary

- Prototyping is thinking, not execution; you build because you have not decided, and the prototype is designed to be thrown away.
- The cost of a change rises by roughly an order of magnitude at each stage, while the freedom to make it falls — so learning is bought early, when it is cheap to act on.
- A prototype is an experiment with a question attached; the question and the observation that would answer it are written before any material is cut.
- Houde and Hill's three dimensions — role, look and feel, implementation — must be named before building; one prototype answers one of them.
- Fidelity is chosen, not maximised. Roughness invites structural criticism; polish invites cosmetic politeness and makes the team protective.
- Sketching is a thinking tool, not a drawing skill: big, fast, labelled, with a human figure for scale, and six variants rather than one careful image.
- A storyboard tests role in six frames; the first and last frames carry the argument. A scenario does the same in half a page about one named person.
- Mock-ups answer questions of size, reach and presence; full size and wrong beats quarter scale and accurate.
- Low-fidelity prototyping uses paper, card and found materials under a timer, builds only what the question touches, and is tested the same day.
- In paper prototyping a team member acts as the computer and may not speak; every hesitation is a finding.
- A Wizard of Oz prototype fakes the machinery and runs the experience for real, and every participant is debriefed afterwards.
- Rapid prototyping runs CAD to mesh to slice to build to post-process to test; FDM, SLA, SLS, metal printing, laser cutting and CNC each suit different questions.
- Rapid is relative and a printed part is not a production part: print what must be geometrically right and build everything else in card.

- Five users per round find about four-fifths of the faults; three rounds of five with changes between them beat one round of fifteen.
- The moderator hands over the prototype and stays silent; explaining converts a test into a sales pitch and a finding into a compliment.
- The feedback capture grid holds likes, criticisms, questions and ideas, recorded verbatim during the session, with behaviour counted rather than opinions.
- Build-measure-learn loops back to the prototype, to ideation or to empathy depending on what the test showed, and each loop should be faster than the last.
- Improvements are ordered by frequency and cost; simplification beats addition, and stopping is a decision recorded with a reason.

17. Review questions

17.1 Short-answer (two-mark) questions

1. Define a prototype as the term is used in design thinking.
2. What is meant by “build to think”?
3. Sketch the cost-of-change curve and explain its shape in one sentence.
4. State the two curves that together justify early prototyping.
5. Give three reasons for building a prototype.
6. Why must a question be written before a prototype is built?
7. Name Houde and Hill's three dimensions of prototyping.
8. Which dimension does a storyboard test, and why?
9. Define fidelity. Is a high-fidelity prototype a better prototype?
10. What is the politeness effect and how does fidelity cause it?
11. Distinguish sketching from drawing.
12. State three rules for a useful design sketch.
13. What goes in the first and last frames of a storyboard?
14. Distinguish a scenario from a feature list with one example of each.
15. Why is a full-size inaccurate mock-up often better than an accurate scale model?
16. List six items in a low-fidelity prototyping toolkit.
17. In a paper prototype, why may the human computer not speak?
18. What is a Wizard of Oz prototype? Give one example.
19. State the ethical obligation that follows a Wizard of Oz test.
20. List the six steps of the CAD-to-part pipeline.
21. Distinguish FDM from SLA in one sentence each.
22. Why is a 3D-printed part a poor guide to the strength of the production part?
23. How many users are tested per round, and on what evidence?
24. State three things a moderator must not do during a test.
25. Name the four quadrants of the feedback capture grid.
26. State the four steps of the build-measure-learn loop.
27. Give two honest conditions for stopping iteration.

17.2 Long-answer (thirteen-mark) questions

1. Explain why prototyping is described as a way of thinking rather than a stage of execution. Discuss the cost-of-change curve and the knowledge-freedom curve, and show how the two together justify early, rough prototyping.
2. Explain Houde and Hill's model of what prototypes prototype. Describe the three dimensions with an example prototype for each, and discuss why a prototype attempting all three usually answers none of them.
3. Discuss fidelity as a design decision. Compare low- and high-fidelity prototypes on cost, the kind of feedback each draws and the risks each carries, and state the rules for choosing a fidelity.
4. Explain concept visualisation. Describe sketching as a thinking tool, list the rules of a useful design sketch, and compare five other instruments of visualisation with the questions each answers.
5. Describe the construction of a six-frame storyboard. Explain the function of each frame and discuss why a storyboard that begins with the product is an advertisement rather than a design argument.
6. Describe low-fidelity prototyping in practice: the toolkit, the method, the finishing condition and the five common faults. Illustrate with a prototype you have built.
7. Explain paper prototyping for an interface, including the role of the human computer. Discuss what is learnt from hesitation and why the team may not explain the prototype during the session.
8. Explain the Wizard of Oz technique with its origin and two worked examples. Discuss what it makes testable that could not otherwise be tested, and the ethical obligations it creates.
9. Describe the rapid-prototyping pipeline from CAD model to tested part. Compare FDM, SLA, SLS, laser cutting and CNC machining, stating what each is suited to.
10. Discuss the limits of rapid prototyping. Explain why "rapid" is relative and why a printed part answers questions of geometry far better than questions of strength.
11. Explain how a user test is planned and moderated. Discuss the evidence for testing about five users per round, the task-based script, the roles in the session and the conduct of the moderator.
12. Explain the feedback capture grid. Describe the four quadrants, the rules for capturing feedback honestly, and how a completed set of grids is read for findings.
13. Explain the iterative design process. Describe the build-measure-learn loop and discuss how a team decides whether a test result sends it back to the prototype, to ideation or to empathy.
14. Discuss design improvement strategies. Explain fix, simplify, combine, reframe, pivot and abandon with an example of each, and state the conditions under which iteration should stop.

Text: Tim Brown, *Change by Design*, Harper Collins, 2012 — the chapters on building to think and on prototyping in the field; Idris Mootee, *Design Thinking for Strategic Innovation*, Wiley, 2013 — Chapter 6. **References:** Stephanie Houde and Charles Hill, "What do Prototypes Prototype?", in *Handbook of Human-Computer Interaction*, Elsevier, 1997; Tom Kelley, *The Art of Innovation*, Doubleday, 2001; Jakob Nielsen and Thomas Landauer, "A Mathematical Model of the Finding of Usability Problems", *Proceedings of ACM INTERCHI*, 1993; Steve Krug, *Rocket Surgery Made Easy*, New Riders, 2010; Nigel Cross, *Engineering Design Methods*, 4th Edn., Wiley, 2008; Jeanne Liedtka and Tim Ogilvie, *Designing for Growth*, Columbia Business School Publishing, 2011 — the chapters on rapid prototyping and the learning launch.
